

Rapport Technique

API Comptabilité Matière

Hôpital Fultang – Module de Gestion des Stocks

École Nationale Supérieure Polytechnique de Yaoundé (ENSPY)
Année Académique 2025–2026

Équipe de Développement – DeDjomo

01 Mai 2026

Table des matières

Première partie

Implémentation

Chapitre 1

Technologies Utilisées

1.1 Langage de programmation

Python 3.x a été retenu comme langage principal. Sa syntaxe claire, son écosystème riche en bibliothèques scientifiques et web, ainsi que sa large adoption dans le milieu académique et hospitalier en font un choix naturel pour un projet de gestion de stocks.

1.2 Framework

- **Django 4.2.7** – Framework web full-stack offrant un ORM puissant, un système de migrations automatiques et une interface d’administration intégrée.
- **Django REST Framework (DRF) 3.14.0** – Extension de Django pour la construction d’API RESTful. Fournit la sérialisation, les ViewSets, le routage automatique et la gestion des permissions.
- **djangorestframework-simplejwt 5.3.1** – Authentification par JSON Web Token (JWT).
- **drf-spectacular 0.27.2** – Génération automatique de la documentation OpenAPI 3.0 (Swagger UI et ReDoc).
- **django-filter 23.5** – Filtrage déclaratif des querysets via les paramètres d’URL.
- **django-cors-headers 4.2.0** – Gestion des en-têtes CORS pour la communication avec le frontend.

1.3 Base de données

PostgreSQL (via **psycopg2-binary 2.9.7**) a été choisi pour sa robustesse, son support des transactions ACID, ses performances en environnement de production et sa compatibilité native avec Django.

Configuration BD

```
DB_NAME=comptamatiere
DB_USER=nehemie
DB_HOST=localhost
DB_PORT=5432
```

1.4 Outils de développement

- **Git** – Contrôle de version distribué.
- **Postman** – Tests manuels et documentation des endpoints API.
- **Gunicorn 21.2.0** – Serveur WSGI pour le déploiement en production.
- **python-dotenv** / **django-environ** – Gestion sécurisée des variables d’environnement.

Chapitre 2

Architecture du Système

2.1 Organisation en couches

L'API suit une architecture en couches stricte, conforme au patron *Model-Serializer-View-URL* de Django REST Framework :

Structure du projet

```
comptamatiere/  
  core/                                # Configuration Django  
    settings.py  
    urls.py  
    wsgi.py  
  apps/  
    comptabilite_matiere/  
      models/                          # Couche Données (13 modèles)  
      serializers/                     # Couche Sérialisation  
      views/                           # Couche Logique (ViewSets)  
      tests/                           # Suite de tests  
      permissions.py                  # Contrôle d'accès  
      urls.py                         # Routage (DefaultRouter)  
  manage.py  
  requirements.txt
```

2.2 Description de l'architecture

1. **Couche Models** : 13 modèles Django organisés en fichiers séparés (héritage `Materiel` → `MaterielMedical`, `MaterielDurable`). Contient la logique métier critique (mise à jour automatique des stocks dans les méthodes `save()`).
2. **Couche Serializers** : Sérialiseurs dédiés par action (Create, Update, Read) pour chaque ressource. Validation des données entrantes.
3. **Couche Views** : ViewSets DRF avec actions personnalisées (`@action`). Chaque ViewSet utilise `get_serializer_class()` pour router vers le bon sérialiseur.
4. **Couche URLs** : Routage automatique via `DefaultRouter` de DRF. Préfixe global : `/api/compta_matiere/`.

2.3 Mode de communication

- **Protocole** : REST over HTTP/HTTPS
- **Format** : JSON (entrée et sortie)
- **Authentification** : JWT (Bearer Token) via `simplejwt`
- **Documentation** : Swagger UI (`/api/docs/`), ReDoc (`/api/redoc/`)

Chapitre 3

Modules Développés

3.1 Module Gestion des Matériels

3.1.1 Matériel Base (/api/compta_matiere/materiels/)

CRUD complet + action `stock_faible` (matériels avec stock < 10).

3.1.2 Matériel Médical (/api/compta_matiere/materiels-medicaux/)

CRUD + actions : `par_categorie`, `stock_faible` (seuil < 20), `alertes_peremption`.
Catégories : MEDICAMENT, CONSOMMABLE, REACTIF. Calcul automatique de la marge bénéficiaire.

3.1.3 Matériel Durable (/api/compta_matiere/materiels-durables/)

CRUD + actions : `mettre_en_reparation`, `remettre_en_service`, `en_reparation`, `par_localisation`. États : BON_ETAT, EN_REPARATION, REFORME.

3.2 Module Gestion des Besoins

Endpoint : /api/compta_matiere/besoins/

Flux complet d'émission et traitement des besoins par le personnel :

- Création avec lignes de besoin (`LigneBesoin`)
- Modification du statut : NON_TRAITE → EN_COURS → TRAITE / REJETE
- Ajout de commentaire du directeur (action `ajouter_commentaire`)
- Consultation personnelle (`mes_besoins`) et groupement par statut (`par_statut`)

Date de traitement automatiquement renseignée lors du passage au statut TRAITE ou REJETE.

3.3 Module Gestion des Stocks (Entrées / Sorties)

3.3.1 Livraisons (/api/compta_matiere/livraisons/)

Enregistrement des entrées de stock. Chaque `LigneLivraison` incrémente automatiquement le `quantite_stock` du matériel associé. Le `montant_total` de la livraison est

recalculé automatiquement. Actions : `par_fournisseur`, `statistiques`.

3.3.2 Sorties (/api/compta_matiere/sorties/)

Enregistrement des sorties. Chaque `LigneSortie` décrémente le stock avec vérification de disponibilité (lève `ValueError` si stock insuffisant). Calcul automatique du sous-total et du montant total. Motifs : `VENTE`, `UTILISATION_SERVICE`, `DEFECTUEUX`, `PERIME`, `PERTE`. Actions : `par_motif`, `mes_sorties`, `statistiques`.

3.4 Module Inventaire

Endpoint : `/api/compta_matiere/archives-inventaire/`

Gestion des inventaires périodiques :

- Création d’une archive avec saisie des stocks réels via `LigneArchiveInventaire`
- Mise à jour par lot (`batch_update`) des lignes d’archive
- Clôture de l’inventaire (`terminer`) : met à jour les stocks théoriques
- Consultation des écarts (`differences`), ancien/nouveau stock

3.5 Module Rapports et Communication

Endpoint : `/api/compta_matiere/rapports/`

Système de messagerie interne entre services :

- Envoi de rapports avec pièces jointes (`PieceJointeRapport`)
- Marquage comme lu (`mark-read`)
- Filtrage : envoyés (`sent`), reçus (`received`), non lus (`received/unread`)
- Types : `GENERAL`, `INVENTAIRE`, `STOCK`, `LIVRAISON`

3.6 Gestion des erreurs et authentification

- **Authentification** : JWT via `simplejwt`. En mode développement (`DEBUG=True`), la permission `DevelopmentOrAuthenticated` autorise l’accès sans token.
- **Validation** : DRF valide automatiquement les données via les sérialiseurs. Les validateurs Django (`MinValueValidator`, `RegexValidator`) protègent l’intégrité au niveau modèle.
- **Erreurs métier** : `ValueError` levée pour les stocks insuffisants. Codes HTTP standard (400, 404, 201, 200).

Chapitre 4

Choix Techniques

4.1 Pourquoi Django et DRF ?

- ORM intégré avec migrations automatiques, idéal pour un schéma de données complexe (13 modèles, héritage).
- DRF fournit des ViewSets, un routage automatique et une sérialisation puissante, réduisant considérablement le code boilerplate.
- Écosystème mature : documentation Swagger automatique (drf-spectacular), filtrage (django-filter), authentification JWT.
- Alternative considérée : Flask + SQLAlchemy. Rejeté car nécessite plus de configuration manuelle pour atteindre le même niveau de fonctionnalité.

4.2 Pourquoi PostgreSQL ?

- Support natif des transactions ACID, critique pour la cohérence des mouvements de stock.
- Support des champs JSON natifs (utilisé pour `PieceJointeRapport.donnees_json`).
- Performances supérieures à SQLite pour les requêtes complexes (agrégations, jointures).
- Compatibilité native avec Django ORM.

4.3 Pourquoi cette structure modulaire ?

La séparation des modèles, sérialiseurs et vues en fichiers individuels favorise :

- La lisibilité et la maintenance du code.
- Le travail collaboratif (merge conflicts réduits).
- La testabilité (chaque module est testable indépendamment).

Chapitre 5

Exemples de Requêtes / Réponses API

5.1 Créer un matériel

POST /api/compta_matiere/materiels/

```
1 // Requete
2 {
3   "code_materiel": "MED-042",
4   "nom_Materiel": "Paracetamol 500mg",
5   "prix_achat_unitaire": "1500.00",
6   "quantite_stock": 200
7 }
8
9 // Reponse (201 Created)
10 {
11   "idMateriel": 42,
12   "code_materiel": "MED-042",
13   "nom_Materiel": "Paracetamol 500mg",
14   "prix_achat_unitaire": "1500.00",
15   "quantite_stock": 200,
16   "date_derniere_modification": "2026-05-01T15:30:00Z"
17 }
```

5.2 Enregistrer une livraison

POST /api/compta_matiere/livraisons/

```
1 // Requete
2 {
3   "bon_livraison_numero": "BL-2026-001",
4   "nom_fournisseur": "Pharma Cameroun",
5   "contact_fournisseur": "677123456",
6   "date_reception": "2026-05-01T10:00:00Z",
7   "montant_total": "0.00"
8 }
9
10 // Reponse (201 Created)
11 {
12   "idLivraison": 1,
13   "bon_livraison_numero": "BL-2026-001",
```

```
14  "nom_fournisseur": "Pharma Cameroun",
15  "montant_total": "0.00"
16 }
```

5.3 Effectuer une sortie (erreur stock insuffisant)

POST /api/compta_matiere/lignes-sortie/ (400)

```
1 // Requete (stock disponible = 5)
2 {
3   "id_sortie": 1,
4   "id_materiel": 42,
5   "code_materiel": "MED-042",
6   "nom_materiel": "Paracetamol 500mg",
7   "type_materiel": "MEDICAL",
8   "quantite": 100
9 }
10
11 // Reponse (400 Bad Request)
12 {
13   "detail": "Stock insuffisant pour Paracetamol 500mg.
14             Disponible: 5"
15 }
```

5.4 Modifier le statut d'un besoin

PATCH /api/compta_matiere/besoins/1/modifier_statut/

```
1 // Requete
2 { "statut": "TRAITE" }
3
4 // Reponse (200 OK)
5 {
6   "message": "Statut modifie avec succes en 'Traite'.",
7   "besoin": {
8     "idBesoin": 1,
9     "statut": "TRAITE",
10    "date_traitement_directeur": "2026-05-01T16:00:00Z"
11  }
12 }
```

Deuxième partie

Tests

Chapitre 6

Types de Tests Réalisés

6.1 Tests unitaires

Fichiers : `test_models.py`, `test_materiel.py`, `test_besoins.py`

- Représentation textuelle (`__str__`) des modèles
- Valeurs par défaut (statut `NON_TRAITE` pour `Besoin`)
- Validation des contraintes (stock négatif interdit)
- Héritage des modèles (`MaterielMedical` hérite de `Materiel`)

6.2 Tests d'intégration

Fichier : `test_api_views.py`

- Requête GET sur la liste des matériels (code 200)
- Requête POST pour créer un matériel (code 201)
- Requête POST avec données invalides (code 400)
- Authentification via `force_authenticate`

6.3 Tests fonctionnels (logique métier)

Fichiers : `test_logic.py`, `test_livraisons.py`, `test_sorties.py`

- Flux complet d'entrée de stock (livraison → incrémentation)
- Flux complet de sortie de stock (sortie → décrémentation)
- Protection contre le stock négatif (`ValueError`)
- Calcul automatique du montant total d'une livraison
- Ajustement du stock lors de la modification d'une livraison

Chapitre 7

Outils de Test

- **Django TestCase** (`django.test.TestCase`) – Framework de test intégré à Django. Fournit une base de données de test isolée avec rollback automatique par transaction. Choisi car natif et parfaitement intégré à l'ORM.
- **DRF APITestCase** (`rest_framework.test.APITestCase`) – Extension de TestCase pour les tests d'API REST. Fournit un client HTTP de test avec support de l'authentification (`force_authenticate`). Choisi car il simule des requêtes HTTP réelles sans serveur.
- **Postman** – Utilisé pour les tests manuels exploratoires et la documentation visuelle des endpoints. Choisi pour sa facilité d'utilisation et le partage de collections entre développeurs.

Chapitre 8

Scénarios de Test

8.1 Gestion des matériels

Cas nominal

Création d'un matériel avec code unique, nom, prix et stock → Objet créé avec `__str__` correct.

Cas d'erreur

Tentative de définir un stock négatif → `ValidationError` levée par `full_clean()`.

8.2 Flux de livraison (entrée de stock)

Cas nominal

Ajout d'une ligne de livraison de 100 unités → Stock du matériel augmente de +100.

Cas limite

Modification de la quantité livrée de 50 à 80 → Stock ajusté de +30 (différence calculée).

8.3 Flux de sortie (sortie de stock)

Cas nominal

Sortie de 30 unités (stock initial = 100) → Stock diminue à 70.

Cas d'erreur

Tentative de sortie de 150 unités (stock = 100) → `ValueError` : "Stock insuffisant".

8.4 Calcul automatique des totaux

Cas nominal

Deux lignes de livraison : $(2 \times 1500) + (3 \times 2000) = 9000$ FCFA $\rightarrow$ `montant_total` = 9000.00.

8.5 API REST

Cas nominal

GET `/api/compta_matiere/materiels/` $\rightarrow$ Code 200, liste non vide. POST avec données valides $\rightarrow$ Code 201, objet persisté.

Cas d'erreur

POST avec `prix_achat_unitaire` = -10 $\rightarrow$ Code 400 Bad Request.

Chapitre 9

Résultats Obtenus

Tous les tests ont été exécutés avec succès. Aucune régression détectée.

Commande : `python manage.py test apps.comptabilite_matiere.tests
--verbosity=2`

Chapitre 10

Tableau Récapitulatif des Tests

N°	Fonctionnalité	Type	Outil	Résultat	Observations
1	Représentation textuelle Materiel	Unitaire	TestCase	Succès	__str__ conforme
2	Stock négatif interdit	Unitaire	TestCase	Succès	ValidationError levée
3	Statut par défaut Besoin	Unitaire	TestCase	Succès	NON_TRAITE
4	Héritage Materiel-Medical	Unitaire	TestCase	Succès	Champs hérités OK
5	Création matériel base	Unitaire	TestCase	Succès	Persistance OK
6	Entrée de stock (livraison)	Fonctionnel	TestCase	Succès	+100 unités
7	Sortie de stock	Fonctionnel	TestCase	Succès	-30 unités
8	Stock insuffisant	Fonctionnel	TestCase	Succès	ValueError levée
9	Calcul montant total	Fonctionnel	TestCase	Succès	9000.00 FCFA
10	Modification livraison	Fonctionnel	TestCase	Succès	Ajustement delta
11	Sortie diminue stock	Fonctionnel	TestCase	Succès	100 → 70
12	Erreur sortie excessive	Fonctionnel	TestCase	Succès	ValueError
13	GET liste matériels	Intégration	APITestCase	Succès	HTTP 200
14	POST créer matériel	Intégration	APITestCase	Succès	HTTP 201
15	POST données invalides	Intégration	APITestCase	Succès	HTTP 400